

Instance Variable & Instance Method

Recap...

As previously discussed, objects can have:

1. Data/ State : Instance Variable
2. Behaviour : Instance method

What is an Instance Variable?

- Declared inside a class.
- Defines the features or attributes of an object.
- Every object of a class will have its own copy of instance variables in the memory.
- Values of those attributes may differ from object to object.
- Can be accessed inside a class easily without using any keyword or by using “this” keyword when needed.
- Can be accessed using object reference outside the design class. Ex: box1.height
(shown in slide 5)

Instance Variable Declaration (Contd.)

Class Design

```
1.  public class ClassName
2.  {
3.      Access_modifier type instance_variable1;
4.      Access_modifier type instance_variable2;
5.      Access_modifier type instance_variable3;
6.  }
```

- Each object of ClassName type will have its own copy of these 3 instance variables.
- There can be more number of instance variables of different types, even an array.

Instance Variable Declaration (Contd.)

Design Class

```
1.  public class Box
2.  {
3.      public double height;    //instance variable
4.      public double width;    //instance variable
5.      public double depth;    //instance variable
6.  }
```

Welcome to DrJava. Working directory is /home/Java111

> run `Tester`

0.0

0.0

0.0

10.2

11.5

13.0

Tester Class

```
1.  public class Tester{
2.      public static void main(String [] args){
3.          Box box1 = new Box();
4.          System.out.println(box1.height);
5.          System.out.println(box1.width);
6.          System.out.println(box1.depth);
7.          box1.height = 10.2;
8.          box1.width = 11.5;
9.          box1.depth = 13.0;
10.         System.out.println(box1.height);
11.         System.out.println(box1.width);
12.         System.out.println(box1.depth);
13.     }
14. }
```

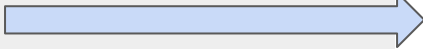
Instance Variable Declaration (Contd.)

- Note that we need to compile the Design class first, then compile and run the Tester class.
- The Design class is not run as it does not have any main method.
- Initially, we only declared 3 variables - height, width, depth but did not assign it any values. So when we create an object, by default, the initial values are 0.0.
- To assign values to the instance variables of box1 object, one approach would be to update it manually in the tester class as shown in the previous example.
- Change to instance variable of one object will not affect the instance variable of another object.

Instance Variable Declaration (Contd.)

```
Box box1 = new Box();  
System.out.println(box1.height);  
System.out.println(box1.width);  
System.out.println(box1.height);
```

Output

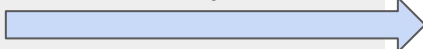


Initial Values of the Instance Variables
when object is created:

box1.height	box1.width	box1.depth
0.0	0.0	0.0

```
box1.height = 10.2;  
box1.width = 11.5;  
box1.depth = 13.0;  
System.out.println(box1.height);  
System.out.println(box1.width);  
System.out.println(box1.height);
```

Output



Updated Values of the Instance Variables

box1.height	box1.width	box1.depth
10.2	11.5	13.0

Instance Variable Declaration (Contd.)

Design Class

```
1. public class Box
2. {
3.     public double height = 10.2;
4.     public double width = 11.2;
5.     public double depth = 13.5;
6. }
```

Set as default
values for all
objects.

Welcome to DrJava. Working directory is /home/Java110

> run `Tester`

10.2

11.2

13.5

10.2

11.2

13.5

Tester Class

```
1. public class Tester{
2.     public static void main(String [] args){
3.         Box box1 = new Box();
4.         Box box2 = new Box();
5.         System.out.println(box1.height);
6.         System.out.println(box1.width);
7.         System.out.println(box1.depth);
8.         System.out.println(box2.height);
9.         System.out.println(box2.width);
10.        System.out.println(box2.depth);
11.    }
12. }
```


What is an Instance Method?

- A certain functionality that needs to be called on that object.
- Can be parameterized or non parameterized.
- Can be used to access or update the instance variables of an object.
- Inside the instance method, we can directly access the instance variable by mentioning its variable name or by using the this keyword.

Instance Method Declaration

- **Notice:** No static keyword as an instance method is not a static method.
- Follows similar declaration rules and signature as a static method.
- Biggest difference is that we did not need an object to call a static method.

```
access_modifier return_type instance_method_name (parameter_type parameter_name) {  
    method body;  
    return return_value; //optional  
}
```

Instance Method Call

- We can call any instance method with respect to an object reference.
- Dot operator “.” is required.
- A parentheses is needed after the method name like it was needed while calling static methods as well.

The syntax for calling an instance method is shown in the box below:

```
object_reference.instance_method_to_be_called()
```

Example 1: Calculating and printing the volume by calling an instance method on that object's reference.

```
1.  public class Box
2.  {
3.      public double height;    //instance variable
4.      public double width;     //instance variable
5.      public double depth;    //instance variable
6.      public void volume()
7.      {
8.          System.out.println("Volume of the box is: " + (height*width*depth));
9.      }
10. }
```

```
1.  public class Tester{
2.      public static void main(String [] args){
3.          Box box1 = new Box();
4.          box1.height = 10.2;
5.          box1.width = 11.5;
6.          box1.depth = 13.0;
7.          box1.volume();
8.      }
9.  }
```

Welcome to DrJava. Working directory is /home/Java111

> run **Tester**

Volume of the box is: 1524.8999999999999

Example 2: Updating the instance variable of an object

```
1.  public class Box
2.  {
3.      public double height;    //instance variable
4.      public double width;    //instance variable
5.      public double depth;    //instance variable
6.      public void update_height(double new_height)
7.      {
8.          height = new_height;
9.      }
10. //Keep the volume() method from previous slide here too.
11. }
```

```
1.  public class Tester{
2.      public static void main(String [] args){
3.          Box box1 = new Box();
4.          box1.height = 10.2;
5.          box1.width = 11.5;
6.          box1.depth = 13.0;
7.          box1.volume();
8.          box1.update_height(22.2);
9.          box1.volume();
10.     }
11. }
```

Welcome to DrJava. Working directory is /home/Java111

> run `Tester`

Volume of the box is: 1524.8999999999999

Volume of the box is: 3318.8999999999996

Array as argument

- The values that we pass to a function can be of any datatype.
- If a non-primitive datatype like an array is passed as arguments the address of that array is received as parameters.
- This is called pass by reference.

```
1. public class OBJ{
2.     public void changeElem(int [] arr){
3.         arr[1] = 12;
4.     }
5. }
```

```
1. public class Tester{
2.     public static void main(String [] args){
3.         OBJ obj1= new OBJ();
4.         int [] array1 = {34, 0 , 65, 7};
5.         System.out.println(array1[1]);
6.         obj1.changeElem(array1);
7.         System.out.println(array1[1]);
8.     }
9. }
```

Welcome to DrJava. Working directory is /home/Java111

> run `Tester`

0

12

Object as argument

- An object can be passed as an argument of any method as well.
- To receive this as parameter the datatype will be the name of the class the object belongs to.
- This is also a form of pass by reference i.e. the address is passed to receiving variable.

```
1. public class Person{
2.     public String role = "Manager";
3.     public void changeRole(Person p){
4.         p.role = "Senior Employee"; //p contains the
5.     }                               //reference/address
6. }                                  //of p2 object.
```

```
1. public class Tester{
2.     public static void main(String [] args){
3.         Person p1 = new Person();
4.         Person p2 = new Person();
5.         p1.changeRole(p2);
6.         System.out.println(p1.role);
7.         System.out.println(p2.role);
8.     }
9. }
```

Welcome to DrJava. Working directory is /home/Java111

> run `Tester`

`Manager`

`Senior Employee`

Object as argument

- To use variable of the instance object which called the method initially, we can directly use the variable name or `this.variable_name`

```
1. public class Person{
2.     public String role = "Manager";
3.     public void compareRole(Person p){
4.         if(role==p.role){ //role or this.role denotes the
5.             System.out.println("Same"); //object that called
6.         } //method
7.     }
8.     else{
9.         System.out.println("Different");
10.    }
11. }
```

```
1. public class Tester{
2.     public static void main(String [] args){
3.         Person p1 = new Person();
4.         Person p2 = new Person();
5.         p1.compareRole(p2);
6.         p1.role = "Employee";
7.         p2.compareRole(p1);
8.     }
9. }
```

Welcome to DrJava. Working directory is /home/Java111

> run `Tester`

Same

Different

Return from an instance method()

- We can return values from inside a function to the main method()
- Based on datatype of the variable returned, the “**return_type**” of the method definition changes. When nothing is returned the type is “void”.
- “**return**” keyword is used to return the value to the line of method call.
- The “**return**” keyword denotes the end of a method() and all lines afterwards is unreachable.

```
1. public class OBJ
2. {
3.     public void add(int a, int b){ //add() without return
4.         int sum = a+b;
5.         System.out.println(sum);
6.     }
7.     public int add(int a, int b){ //add() with return
8.         int sum = a+b;
9.         return sum;
10.    }
11. }
```

```
1. public class Tester
2. {
3.     public static void main(String [] args)
4.     {
5.         OBJ obj1= new OBJ();
6.         obj1.add(5,8); //calls add without return
7.         System.out.println(obj1.add(4,7)); //calls add with return
8.         int return_value = obj1.add(4,7); //calls add with return
9.         System.out.println(return_value);
10.    }
11. }
```

Use of “this.”

- A variable created inside an instance method() of a class is the local variable.
- To modify instance variable from an instance method() with the same name as its local variable, “this” keyword is used.
- “this” holds the location of an object.

```
1. public class Box
2. {
3.     public double height;      //instance variable
4.     public void update_height(double height)
5.     {
6.         this.height = height; //modifying instance variable height
7.     }
8. }
```

```
1. public class Tester{
2.     public static void main(String [] args){
3.         Box box1 = new Box();
4.         box1.height = 10.2;
5.         System.out.println(box1.height); //prints 10.2
6.         box1.update_height(22.2);
7.         System.out.println(box1.height); //prints 22.2
8.     }
9. }
```

THANK YOU!